JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

# Introduction to Generative AI

## Text Generation

# Outline

- Module Learning Objectives
- N-Grams
- Naive Bayes Text Generation
- Transformers
- Large Language Models
- LLM Problems
- LangChain Framework

# Module Learning Objectives

- Explain n-gram and Naive Bayes models for text generation
- Analyze the role of transformers in text generation and compare them to classical methods
- Discuss the strengths, limitations, and challenges of large language models
- Identify causes of hallucination in text generation and propose mitigation strategies
- Demonstrate chaining components in LangChain for advanced text generation tasks

# Text Generation

- A fundamental task in natural language processing (NLP) and AI
- Machines to produce human-like text for translation, chatbots, and content creation
- **N-gram models** predict the next word in a sequence based on the prev n-1 words
  - by **Markov assumption** $\mathrm{P}(w_1, \dots, w_N) \approx \prod_{i=1}^{N} \mathrm{P}(w_i \mid w_{i-(n-1)}, \dots, w_{i-1})$
  - Captures local context and is computationally efficient
- **Naive Bayes** compute probabilistic word relationships to help generate text
- **Transformers** via attention mechanisms capture long-range dependencies
- **Large Language Models** (LLMs) via transformers such as OpenAI GPT-3
  - Pre-trained on vast data for a wide array of tasks with minimal fine-tuning
- **LangChain** can chain together multiple language models and processing steps

# N-gram Models for Text Generation

- A probabilistic language model that predicts the next word in a sequence
- Example, a bigram model (n=2)

$$\mathrm{P}(W) \approx \prod_{i=1}^{N} \mathrm{P}(w_i \mid w_{i-1})$$

- Use delimiters <s> and </s> to mark start and end of sequences
- Process the sequences by moving a window of length n from left to right
- Compute frequencies and conditional probabilities
- Optionally smooth, in case a large corpora is used, by Laplace smoothing, etc.
- Generate text until a delimiter </s> is generated
  - Common iterative approach that can apply to any text generation method

# Naive Bayes Models for Text Generation

- A probabilistic model based on applying **Bayes' theorem**
- Primarily used for classification tasks, and can text generate by treating each word as an independent event conditioned on a particular class or context
- Model the probability of a word sequence $W = (w_1, \dots, w_N)$ given a class $C$
  - Can represent a topic, sentiment, etc.

$$\mathrm{P}(W|C) \approx \prod_{i=1}^{N} \mathrm{P}(w_i \mid C)$$

- **NB assumption** each $w_i$ is conditionally independent of other words given $C$
- Sample words sequentially from the distribution $\mathrm{P}(w|C)$ to generate text

# Laplace Smoothing

- Adds a small constant (usually 1) to each count in a probability distribution to avoid zero probabilities for unseen events
- Ensures more robust and generalizable models, particularly in cases with sparse data
- Balances seen and unseen events

$$\mathrm{P}(w|C) = \frac{\mathrm{Count}(w, C) + 1}{N_C + |V|}$$

- $N_C$ The total number of words (tokens) in the class $C$
- $|V|$ The size of the vocabulary (across all classes)

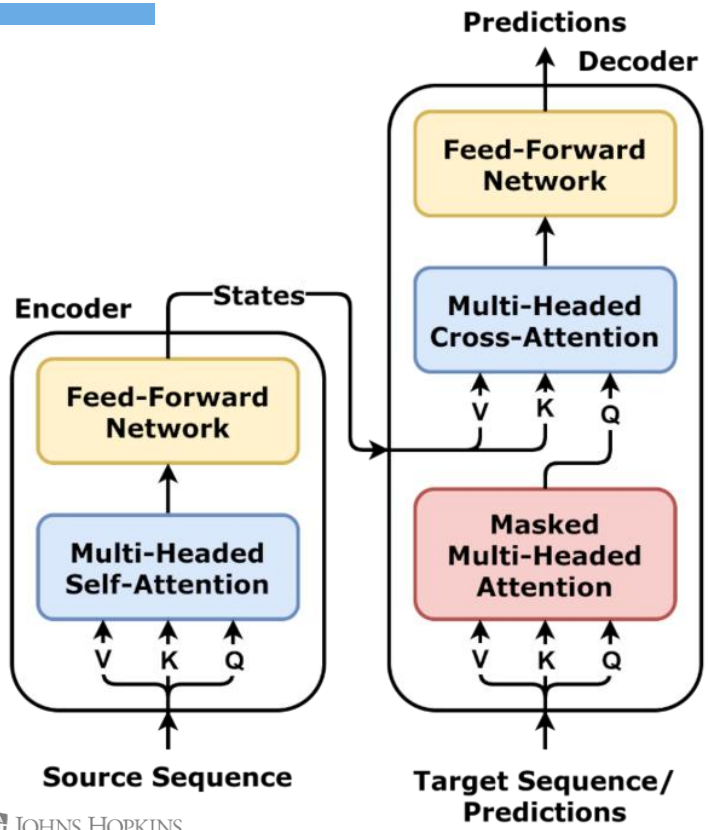
# Transformer for Text Generation

- The **Transformer** model is a sequence-to-sequence architecture that relies entirely on attention mechanisms to draw global dependencies between input and output
- Eliminates recurrence and convolutions, enabling parallelization and more efficiency
  - Unroll the RNN, replace it by general attention
- **Self-attention** mechanism computes a representation of the sequence by relating different positions of the sequence to each other

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\mathrm{T}}}{\sqrt{d_k}}\right)V$$

- $Q$, $K$, $V$ are the query, key and value matrices

# Transformer Architecture




- **Encoder** processes the source seq
  - **Multi-Headed Self-Attn** to learn contextual relationships btw tokens
  - **FF Net** for further transformation
- **Decoder** generates predictions from the target sequence
  - **Masked Multi-Headed Attn** to prevent future token peeking
  - **Multi-Headed Cross-Attn** to attend to encoder outputs
  - **FF Net** to refine predictions
- Source: Illustrations for the Transformer, and attention mechanism. Transformer, one encoder-decoder block. Author dvgodoy. https://github.com/dvgodoy/dl-visuals/?tab=readme-ov-file

# Transformer Implementation

```python
class Encoder(nn.Module):
  def __init__(self, n_inp, n_hid):
    super().__init__()
    self.n_inp, self.n_hid = n_inp, n_hid
    # n_inp=voc size, n_hid=embedding size
    self.emb = nn.Embedding(n_inp, n_hid)
    # RNN layer, used token by token
    self.rnn_cell = nn.GRU(n_hid, n_hid,
      batch_first=True)

  def forward(self, _X, _hn, bs=BATCH_SIZE):
    # single seq input # T x BS x hidden_size
    X_embedded = self.emb(_X)
    output, hn = self.rnn_cell(X_embedded, _hn)
    return output, hn

  def init_hidden(self, bs=BATCH_SIZE):
    # 1 x BS x H by design of PyTorch GRU
    return zeros(1, bs, self.n_hid, device=Dev)
```

```python
class SingleHeadAttention(nn.Module):
  def __init__(self, n_hid):
    super().__init__()
    self.n_hid = n_hid
    # linear layers to project input to Q, K, V
    self.Qlayer = nn.Linear(n_hid, n_hid)
    self.Klayer = nn.Linear(n_hid, n_hid)
    self.Vlayer = nn.Linear(n_hid, n_hid)

  def forward(self, _X):
    # X input embeddings BS x T x n_hid
    # returns context vectors BS x T x n_hid
    Q = self.Qlayer(_X)
    K = self.Klayer(_X)
    V = self.Vlayer(_X)
    attn_scores = matmul(Q, K.transpose(-2,-1))
/ sqrt(tensor(self.n_hid))
    attn_ws = F.softmax(attn_scores, dim=-1)
    context = matmul(attn_ws, V)
    return context, attn_ws
```

# Transformer Decoder

```python
class GeneralAttentionDecoder(nn.Module):
  def __init__(self, n_hid, n_out, n_heads=8,
dropout=0.1):
    super().__init__()
    self.n_hid, self.n_out = n_hid, n_out
    self.embed = nn.Embedding(self.n_out,
      self.n_hid)
    self.dropout = nn.Dropout(dropout)
    # Multi-head attention mechanism
    self.attn =
      nn.MultiheadAttention(
      embed_dim=self.n_hid, num_heads=n_heads,
      batch_first=True)
    self.rnn_cell =
      nn.GRU(input_size=self.n_hid,
      hidden_size=self.n_hid, batch_first=True)
    self.w_y =
      nn.Linear(in_features=self.n_hid,
      out_features=self.n_out)
```

```python
def forward(self, _X, _hn, _encoder_outputs):
  X_embed = self.embed(_X)
  X_embed = self.dropout(X_embed)
  # Compute attention weights and
  # apply attention
  attn_output, attn_weights =
    self.attn(query=_hn.permute(1, 0, 2),
    key=_encoder_outputs,
    value=_encoder_outputs)
  # Combine attention output
  # with decoder input
  rnn_input = attn_output + X_embed
  _, hn = self.rnn_cell(rnn_input, _hn)
  output =
    nn.functional.log_softmax(
    self.w_y(hn.permute(1, 0, 2)), dim=2)
  return output, hn, attn_weights
```

# Large Language Models

- AI models trained on massive text data to understand, generate, and manipulate human language
  - **Context Understanding** grasp meaning from context with coherent responses
  - **Text Generation** create human-like text for various applications
  - **Scalability** larger models improve performance on complex language tasks
- Limitations
  - **Bias** models can inherit biases from training data
  - **Misuse Potential** with fake news, plagiarism, etc.
  - **Resource Intensive** as it requires substantial computational power
  - **Lack of Reasoning** as struggles with logic beyond training data

# LLM for Text Generation

- Large language models are sequence-to-sequence neural network models that are trained on vast amounts of textual data
- They can perform a variety of language tasks, including text generation
- They leverage the **Transformer**, to capture complex patterns in language
- The transformer neural network model aims to estimates the joint probability

$$\mathrm{P}(w_1, w_2 \ldots, w_N) = \prod_{i=1}^{N} \mathrm{P}(w_i \mid w_1, w_2 \ldots, w_{i-1})$$

- **Hyperparameters** of attention heads, hidden dimensions, hidden layers, number of layers, and learning rate
- Generally we use pre-trained models, Generative Pretrained Transformers (GPT)

# Some Negative Aspects of LLMs

- **Computational Expense** require massive computational resources
- **Data Inefficiency** need vast amounts of training data, may not be available, especially for niche domains or languages
- **Bias Propagation** inherit and amplify biases present in the training data
- **Lack of True Understanding** lack real-world understanding
  - Quality versus quantity, IBM Deep Blue is a historical counter-example
- **Opacity** black box models, perhaps like our brains, making it difficult to interpret or explain how they arrive at certain decisions
  - As remedy, some suggestions like attention visualization, integrated gradients, probe tasks and XAI frameworks (LIME, SHAP) proposed

# Hallucination

- Refers to situations where the model generates plausible-sounding but factually incorrect, fabricated, or nonsensical information
- **Hallucination is a Problem** because (1.) it undermines the trust in LLMs, e.g. legal advice, medical diagnoses, research; (2.) the content can spread misinformation, waste human effort in verifying details; (3.) it can lead to severe consequences, e.g. financial losses, legal liability, or safety; (4.) fact-check need negates its utility for saving time or simplifying workflows
- **Causes of Hallucination** include (1.) training data limitations where LLM learns the "patterns" (not what is true or false); (2.) probabilistic nature of the models; (3.) lack of grounding due to lack access to real-time, external knowledge bases or fact-checking mechanisms; (4.) overgeneralization of patterns seen in training; (5.) ambiguity in prompts as the model may try to "fill in the blanks" creatively rather than factually

# Efforts to Mitigate Hallucination

- **Fact-Checking Pipelines** can validate outputs against trusted sources in real-time
- **Fine-Tuning** on domain-specific data or incorporating feedback loops to reduce hallucinations in specialized contexts
- **External Knowledge Integration** by models like GPT with browsing or plugin capabilities can query up-to-date and reliable sources to ground responses
- **Human-in-the-Loop Systems** help pairing LLMs with human reviewers or moderators for applications where accuracy is critical
- **Post-Processing Filters** can implement post-generation validation checks to flag or correct hallucinations
- Hallucination may be tolerated in creative applications when it allows for novel and imaginative outputs, e.g. such as storytelling, brainstorming, or other creative tasks

# LangChain for Text Generation

- A **framework** designed to facilitate developing LLM applications
- Chain various components
  - LLMs, prompt templates, memory modules, RAG, agents, etc.
- **Prompt Templates** structured templates that can dynamically incorporate variables
- **LLMs** large language models that generate text based on input prompts
- **Chains** of operations such as sequential chains, transforms, and LLMs
- **Memory** to maintain state across interactions, context-aware text generation
- **Retrieval-Augmented Generation** retrieval from external databases or repo
- **Agents** to interact and decide which action to take based on responses

# References


- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Manning, Christopher, and Hinrich Schutze. Foundations of statistical natural language processing. MIT press, 1999.
- Vaswani, A. "Attention is all you need." Advances in Neural Information Processing Systems (2017).

JOHNS HOPKINS

WHITING SCHOOL
*of* ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.